



# ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE — FALL 2021

## HOMEWORK 4

**Due:** 11:59pm, Saturday, April 17

**Submit to:** [engin604assignments@gmail.com](mailto:engin604assignments@gmail.com)

Maximum page limit: 5 pages

See the homework policies at <https://docenciaweb.fen.uchile.cl>



The file `ETFs.xlsx` contains the OHCL (*Open, High, Close and Low*) of the following *Exchange Traded Funds*<sup>1</sup> at a daily frequency:

- SPY** (S&P500 ETF), from 2012-11-26 to 2020-12-30
- EFA** (a non-US equities ETF), from 2012-12-31 to 2020-12-30
- IJS** (a small-cap value ETF), from 2012-10-31 to 2020-12-30
- EEM** (an emerging-markets ETF), from 2012-04-25 to 2020-12-30
- AGG** (A bond ETF), from 2012-12-31 to 2020-12-30

All the data were extracted from *Yahoo Finance* using the `YFinance` library.

- Load the file `ETFs.xlsx` into your workspace so that each sheet is stored in a dictionary.
- Build a `DataFrame` containing the closing price of each ETF.
  - The column (`columns`) must have the name of the ETF and the index (`index`) the date.
  - Work only with the data from 2012-12-31 to 2020-12-30.

Explain at least two ways of doing (2.b).

- Transform the `DataFrame` created in (2) to a monthly frequency. This is done by assuming that the last available price of the month corresponds to the monthly price.
- Using a `lambda` (anonymous) function, generate the monthly logarithmic return for each ETF. Do not forget to drop the row of `NAs`.

$$R_{j,t} = \ln \left( \frac{P_{j,t}}{P_{j,t-1}} \right) \quad j = 1, \dots, 5 \quad t = 1, \dots, 97$$

*Hint:* NumPy has the `np.log()` function to compute the natural logarithm.

- Write a function that computes a brief descriptive statistic containing: arithmetic mean, standard deviation, *kurtosis*, *skewness* and Sharpe ratio (assume a risk-free rate equal to 0). For the function, take the following into account:
  - The *input* must be a Pandas with a Series structure.
  - The *output* must be a Pandas with a Series structure where:

---

<sup>1</sup>These are collective investment funds whose investment policy consists of replicating an index.

- The `name` attribute contains the name of the *input*.
- The `index` attribute contains the name of each descriptive statistic computed.

Apply the function to the DataFrame created in (4) by means of a `lambda` (anonymous) function.

- Using a line chart, plot the cumulative logarithmic return of each ETF.
- The tangency portfolio is the one that maximizes the Sharpe ratio. The optimization problem expressed in its matrix form is:

$$\max_{\mathbf{t}} \frac{\mathbf{t}'\boldsymbol{\mu} - r_f}{(\mathbf{t}'\boldsymbol{\Sigma}\mathbf{t})^{\frac{1}{2}}} = \frac{\mu_{p,t} - r_f}{\sigma_{p,t}} \quad s.t \quad \mathbf{t}'\mathbf{1} = 1$$

Whose solution (also in matrix form) is:

$$\mathbf{t} = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \cdot \mathbf{1})}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \cdot \mathbf{1})}$$

Where  $\mathbf{t}$  is the vector of optimal weights of the tangency portfolio,  $\mathbf{1}$  a vector of ones whose dimension is  $j \times 1$ , with  $j$  the total number of assets, and  $\boldsymbol{\Sigma}^{-1}$  the inverse of the variance-covariance matrix ( $j \times j$ ).

Using the matrix solution for  $\mathbf{t}$ , write a function whose *inputs* are: the vector of average returns, the variance-covariance matrix of the assets (ETFs) that would make up the tangency portfolio and the risk-free rate (scalar).

The *output* of the function must be the weights  $\mathbf{t}$ , the expected return and the standard deviation of the tangency portfolio.

- The expected return of the tangency portfolio is:

$$R_{p,t} = \mathbf{t}' \cdot \boldsymbol{\mu}$$

Where  $\boldsymbol{\mu}$  is the vector of average returns of dimension  $j \times 1$ .

- The standard deviation of the minimum-variance portfolio is:

$$\sigma_{p,t} = \sqrt{\mathbf{t}'\boldsymbol{\Sigma}\mathbf{t}}$$

Test the function using the average returns created in (5) and build the variance-covariance matrix using the DataFrame generated in (4). Explain your function line by line. Assume a risk-free rate equal to 0.005.

*Hint:* To convert rows or columns of a DataFrame to NumPy, use the `.to_numpy()` attribute after the selection (`loc` or `iloc`).